

Feedback

Workshop 2

Workshop 2

- The second workshop looked at how to use OpenMP to parallelise a program which calculated Lissajous curves
- To parallelise the program correctly we need to consider variable scoping and loop carried dependencies

Workshop 2

- Open the program in your chosen text editor and find the for loop in the main function.

Question 1: Are there any loop carried dependencies in this loop which prevent it being parallelised?

```
/* Define parameters of the Lissajous curve as constants*/
const float A=1.0;
const float B=1.0;
const float omega_x=1.0;
const float omega_y=2.0;
const float delta_x=PI/2.0;
const float delta_y=0.0;
```

```
/* Number of (x,y) points to calculate */
const int N=10000;
```

```
/* Arrays to hold values of x and y */
float x[N], y[N];
```

```
/* Parameter t used in the parametric equations */
float t;
```

```
/* Calculate values for x and y by looping over the parameter t */
for (int i=0; i<N; i++){
    /* Convert i and N to floats so we do floating point division */
    t = 2.0*PI * (float) i / (float) N;
    x[i] = A * sin( omega_x*t + delta_x );
    y[i] = B * sin( omega_y*t + delta_y );
}
```

```
/* Write the results to a text file called output.dat */
/* x and y are converted to pointers because they are arrays */
write_to_file("output.dat", x, y, N);
```

```
return 0;
}
```

t depends on i so each loop iteration will have its own value → t should be private so no loop carried dependency

Each loop reads/writes to a different element of arrays x and y so there is no loop carried dependency

Workshop 2

- There is also a for loop in the `write_to_file` function.

Question 2: Can the for loop in the `write_to_file` function be parallelised?

```
void write_to_file(const char *filename, float *x, float *y, int num_pts){  
  
    /* Open the file for writing */  
    FILE *outfile;  
    outfile = fopen(filename, "w");  
  
    /* If the file could not be opened then the pointer to it will be NULL */  
    if (outfile == NULL){  
        printf("ERROR: could not open file %s\n", filename);  
        exit(8);  
    }  
  
    /* Loop through x and y arrays writing out the values to the file */  
    for (int i=0; i<num_pts; i++){  
        fprintf(outfile, "%f %f\n", x[i], y[i]);  
    }  
    fclose(outfile);  
}
```

These print statements need to happen in order
so the loop iterations must take place in order.
We can't parallelise this loop.

Workshop 2

Question 3: How did you scope the variables to make sure the parallel version gives identical output to the serial version?

```
/* Calculate values for x and y by looping over the parameter t */
#pragma omp parallel for default(shared) private(t)
for (int i=0; i<N; i++){
    /* Convert i and N to floats so we do floating point division */
    t = 2.0*PI * (float) i / (float) N;
    x[i] = A * sin( omega_x*t + delta_x );
    y[i] = B * sin( omega_y*t + delta_y );
}
```

Scoping: `default (shared)` and list the exceptions
This avoids relying on the scoping rules in the standard
which someone reading the code may need to look up
(more important for more complex programs)

Workshop 2

5 Generating multiple curves

Question 4: Which loop would you choose to parallelise and why?

```
/* Calculate values for x and y by looping over the parameter t */
for (int curve=0; curve<N_CURVES; curve++){
    for (int i=0; i<N_POINTS; i++){
        /* Convert i and N to floats so we do floating point division */
        t = 2.0*PI * (float) i / (float) N_POINTS;
        x[curve][i] = A[curve] * sin( omega_x[curve]*t + delta_x[curve] );
        y[curve][i] = B[curve] * sin( omega_y[curve]*t + delta_y[curve] );
    }
}
```

- `N_CURVES=4` so parallelising the outer loop means we can't use more than 4 threads
- We could parallelise the inner loop (this would start a parallel region for each iteration of the outer loop though)
- Alternatively we could swap the loops and parallelise the outer loop (now over `i`). This gives correct results (in this case) but affects the memory access (more on this in week 6 ...)

Workshop 2

If you add the clause `collapse (2)` to a `#pragma omp parallel for` directive it will result in the following two loops being combined for parallelisation.

Question 5: Could you use the collapse clause with these loops and does it change your answer to the previous question?

```
/* Calculate values for x and y by looping over the parameter t */
#pragma omp parallel for default(shared) private(t) collapse(2)
for (int curve=0; curve<N_CURVES; curve++){
    for (int i=0; i<N_POINTS; i++){
        /* Convert i and N to floats so we do floating point division */
        t = 2.0*PI * (float) i / (float) N_POINTS;
        x[curve][i] = A[curve] * sin( omega_x[curve]*t + delta_x[curve] );
        y[curve][i] = B[curve] * sin( omega_y[curve]*t + delta_y[curve] );
    }
}
```

Workshop 2

```
/* Calculate values for x and y by looping over the parameter t */
#pragma omp parallel for default(shared) private(t)
for (int curve=0; curve<N_CURVES; curve++){
    for (int i=0; i<N_POINTS; i++){
        printf("Thread %d: curve=%d, i=%d\n", omp_get_thread_num(), curve, i);
        /* Convert i and N to floats so we do floating point division */
        t = 2.0*PI * (float) i / (float) N_POINTS;
        x[curve][i] = A[curve] * sin( omega_x[curve]*t + delta_x[curve] );
        y[curve][i] = B[curve] * sin( omega_y[curve]*t + delta_y[curve] );
    }
}
```

N_CURVES=4

N_POINTS=10

OMP_NUM_THREADS=8

Threads 4-7 are idle →

```
Thread 1: curve=1, i=0
Thread 1: curve=1, i=1
Thread 1: curve=1, i=2
Thread 1: curve=1, i=3
Thread 1: curve=1, i=4
Thread 1: curve=1, i=5
Thread 1: curve=1, i=6
Thread 1: curve=1, i=7
Thread 1: curve=1, i=8
Thread 1: curve=1, i=9
Thread 2: curve=2, i=0
Thread 2: curve=2, i=1
Thread 2: curve=2, i=2
Thread 2: curve=2, i=3
Thread 2: curve=2, i=4
Thread 2: curve=2, i=5
Thread 2: curve=2, i=6
Thread 2: curve=2, i=7
Thread 2: curve=2, i=8
Thread 2: curve=2, i=9
Thread 0: curve=0, i=0
Thread 0: curve=0, i=1
Thread 0: curve=0, i=2
Thread 0: curve=0, i=3
Thread 0: curve=0, i=4
Thread 0: curve=0, i=5
Thread 0: curve=0, i=6
Thread 0: curve=0, i=7
Thread 0: curve=0, i=8
Thread 0: curve=0, i=9
Thread 3: curve=3, i=0
Thread 3: curve=3, i=1
Thread 3: curve=3, i=2
Thread 3: curve=3, i=3
Thread 3: curve=3, i=4
Thread 3: curve=3, i=5
Thread 3: curve=3, i=6
Thread 3: curve=3, i=7
Thread 3: curve=3, i=8
Thread 3: curve=3, i=9
```


Workshop 2

```
/* Calculate values for x and y by looping over the parameter t */
#pragma omp parallel for default(shared) private(t) collapse(2)
for (int curve=0; curve<N_CURVES; curve++){
    for (int i=0; i<N_POINTS; i++){
        printf("Thread %d: curve=%d, i=%d\n", omp_get_thread_num(), curve, i);
        /* Convert i and N to floats so we do floating point division */
        t = 2.0*PI * (float) i / (float) N_POINTS;
        x[curve][i] = A[curve] * sin( omega_x[curve]*t + delta_x[curve] );
        y[curve][i] = B[curve] * sin( omega_y[curve]*t + delta_y[curve] );
    }
}
```

N_CURVES=4

N_POINTS=10

OMP_NUM_THREADS=8

Threads 4-7 are working →

```
Thread 1: curve=0, i=5
Thread 1: curve=0, i=6
Thread 1: curve=0, i=7
Thread 1: curve=0, i=8
Thread 1: curve=0, i=9
Thread 0: curve=0, i=0
Thread 0: curve=0, i=1
Thread 0: curve=0, i=2
Thread 0: curve=0, i=3
Thread 0: curve=0, i=4
Thread 2: curve=1, i=0
Thread 4: curve=2, i=0
Thread 4: curve=2, i=1
Thread 4: curve=2, i=2
Thread 4: curve=2, i=3
Thread 4: curve=2, i=4
Thread 2: curve=1, i=1
Thread 2: curve=1, i=2
Thread 2: curve=1, i=3
Thread 2: curve=1, i=4
Thread 5: curve=2, i=5
Thread 5: curve=2, i=6
Thread 5: curve=2, i=7
Thread 5: curve=2, i=8
Thread 5: curve=2, i=9
Thread 6: curve=3, i=0
Thread 6: curve=3, i=1
Thread 6: curve=3, i=2
Thread 6: curve=3, i=3
Thread 6: curve=3, i=4
Thread 7: curve=3, i=5
Thread 7: curve=3, i=6
Thread 7: curve=3, i=7
Thread 7: curve=3, i=8
Thread 7: curve=3, i=9
Thread 3: curve=1, i=5
Thread 3: curve=1, i=6
Thread 3: curve=1, i=7
Thread 3: curve=1, i=8
Thread 3: curve=1, i=9
```